

Platform Perencanaan Asesmen

Dokumentasi Teknis (TELKOM HCSP)

Dokumentasi teknis untuk Platform Perencanaan Asesmen (TELKOM HCSP). Panduan ini ditujukan bagi pengembang (*developer*) dan *engineer* untuk memahami arsitektur, *tech stack*, struktur direktori, serta endpoint API utama dari sistem ini.

1. Arsitektur Sistem & Tech Stack

Proyek ini menggunakan arsitektur **Monorepo** (dikelola melalui *npm workspaces*), yang memungkinkan bagian *frontend*, *backend*, dan *AI service* berada di dalam satu repositori yang sama.

Frontend (`/apps/web`)

- **Framework:** Next.js 16 (App/Pages router)
- **UI Library:** React 19
- **Styling:** Tailwind CSS 4
- **Icons:** Lucide React
- **Visualisasi Data:** Recharts
- **Komunikasi Real-time:** Socket.IO Client

Backend (`/backend`)

- **Runtime:** Node.js
- **Framework:** Express.js
- **Bahasa:** TypeScript
- **Database ORM:** Prisma
- **Database:** PostgreSQL
- **Mesin Real-time:** Socket.IO (untuk notifikasi dan *broadcasting* mutasi data)
- **Penanganan File:** Multer & ExcelJS (untuk mem-*parsing* unggahan Excel)
- **Dokumentasi API:** Swagger UI (`swagger-jsdoc` , `swagger-ui-express`)

AI Service (`/apps/ai_service`)

- **Runtime/Bahasa:** Python 3
- **Framework:** FastAPI
- **Server:** Uvicorn

- **Mesin AI/LLM:** OpenAI GPT via `GPTRunTime` kustom
- **Tujuan:** Mem-*parsing* dan menganalisis respons karyawan (terima, tolak, jadwal ulang) berdasarkan pemrosesan bahasa alami (NLP).

2. Database Login Bawaan (Default)

Sistem ini menggunakan database PostgreSQL yang dikelola melalui Prisma. Secara bawaan (*default*), terdapat akun admin master yang disiapkan selama pengaturan awal (setelah seeding):

- **NIK / Username (nik_user):** `000000` (atau `admin`)
- **Password:** `password123`
- **No. Handphone:** `0812345678`
- **Email:** `admin@telkom.co.id`
- **Role:** `ADMIN`
- **Talent Solution Level:** `0` (Super Admin)

Catatan: Pengguna dan kata sandi dibuat di dalam direktori `backend/prisma` dan `backend/scripts`.

3. Direktori Penting

Memahami struktur monorepo sangat penting untuk dapat menavigasi basis kode (*codebase*) dengan efektif.

Root Directory (Direktori Utama)

- `package.json`: Mengelola monorepo workspaces (`apps/*` dan `backend`).

Frontend (`/apps/web`)

Berisi antarmuka pengguna (UI) dan logika sisi klien (*client-side logic*).

- `/src` atau `/app` / `/pages`: Berisi rute dan komponen Next.js.
- `package.json`: Dependensi dan skrip khusus untuk *frontend*.

Backend (`/backend`)

Server API inti yang mengelola operasi database dan aturan bisnis.

- `/prisma`: Berisi file `schema.prisma` yang mendefinisikan model database dan relasinya.
- `/src/controllers`: Logika bisnis untuk menangani *request* (contoh: `aiController.ts`, `webhookController.ts`).

- `/src/routes` : Definisi rute Express yang dipetakan ke kontroler (`index.ts` adalah *router* utama).
- `/src/services` : Lapisan layanan untuk aturan bisnis kompleks seperti `employeeStatusService.ts`.
- `/src/middleware` : Middleware Express kustom (contoh: CORS, upload multer).
- `/src/socket.ts` : Mengelola koneksi Socket.IO untuk pembaruan *frontend* secara *real-time*.

AI Service (`/apps/ai_service`)

Sebuah *microservice* yang didedikasikan untuk tugas-tugas AI.

- `main.py` : Titik masuk aplikasi FastAPI yang mendefinisikan *endpoint*.
- `gpt_runtime.py` : Lapisan interaksi (*wrapper*) untuk terhubung dengan OpenAI API.

4. Status Karyawan (Penerima)

Di dalam database, setiap penerima (karyawan) memiliki `availability_status` yang merepresentasikan posisi mereka dalam siklus notifikasi. Status-status tersebut adalah:

1. **No Invitation (Belum Diundang)**: Status bawaan saat karyawan pertama kali diimpor. Mereka belum dikirim pesan *broadcast* WhatsApp.
2. **Pending (Tertunda)**: *Broadcast* WhatsApp telah dipicu, namun pesan mungkin gagal, ditolak oleh penyedia layanan, atau belum berhasil masuk ke perangkat pengguna.
3. **Sent (Ter kirim)**: Pesan WhatsApp berhasil dikirimkan ke perangkat pengguna.
4. **Accepted (Diterima)**: Pengguna telah membalas dan mengonfirmasi bahwa mereka akan hadir pada asesmen.
5. **Rejected (Ditolak)**: Pengguna telah menyatakan secara eksplisit bahwa mereka tidak dapat hadir.
6. **Reschedule Requested (Minta Jadwal Ulang)**: Pengguna meminta untuk mengubah tanggal, baik dengan memberikan tanggal secara eksplisit (misal: "Besok") maupun sekadar meminta tanggal baru.

5. Endpoint API Penting

API Express Backend (Base URL: `http://localhost:8000`)

(Tersedia secara interaktif melalui Swagger UI di `http://localhost:8000/api-docs`)

POST

`/api/auth/login`

Mengautentikasi pengguna dan menerima token.

POST`/api/ai/process`

Memproses data generik melalui AI Service.

GET`/api/data/`

Mengambil data asesmen.

POST`/api/upload-excel`

Mengunggah dan mem-*parsing* file Excel.

Webhook & Notifikasi (Integrasi OCA)

Bagian *backend* menerima *webhook* dari OCA (Omnichannel Assistant) ketika pengguna membalas pesan *broadcast* atau ketika status pengiriman pesan berubah.

1. Webhook Pesan (Message)

POST`/api/webhooks/whatsapp`

Dipicu ketika penerima membalas pesan WhatsApp. Akan memicu analisis AI Service.

Format Payload yang Diharapkan:

```
{
  "from": "6281234567890",
  "to": "6280987654321",
  "timestamp": "2023-10-27T10:00:00Z",
  "message": {
    "content": {
      "text": "IYA",
      "type": "text"
    }
  }
}
```

2. Webhook Status Pengiriman (Delivery Status)

POST

/api/webhooks/delivery

Dipicu oleh OCA untuk memperbarui status pengiriman pesan *broadcast*. Digunakan untuk mengubah status karyawan dari Pending menjadi Sent.

Format Payload yang Diharapkan:

```
{
  "msgid": "msg-12345",
  "phone_number": "6281234567890",
  "status": "delivered",
  "timestamp": "2023-10-27T10:00:05Z",
  "error": {
    "code": "kode error (opsional)",
    "reason": "alasan (opsional)"
  }
}
```

API FastAPI AI Service (Base URL: <http://localhost:8001>)

POST

/analyze-response

- **Payload:** { "employeeNik": "string", "response": "string" }
- **Deskripsi:** Menggunakan GPT untuk mengklasifikasikan respons bahasa alami menjadi: 'accepted', 'rejected', atau 'reschedule'. Mengekstrak tanggal menjadi format DD/MM/YYYY.

6. Panduan Instalasi & Setup

Jika Anda menyiapkan proyek ini di PC baru, harap ikuti langkah-langkah ini secara berurutan.

Prasyarat (Prerequisites)

- **Node.js** (v18 atau lebih tinggi): Diperlukan untuk *Frontend* dan *Backend*.
- **Python** (v3.9 atau lebih tinggi): Diperlukan untuk *AI Service*.
- **PostgreSQL**: Database relasional. Pastikan layanan ini berjalan di mesin Anda (port default 5432).
- **Git**: Version control untuk mengkloning repositori.

Langkah 1: Kloning Repositori

```
git clone <repository_url>
cd tes_nextjs
```

Langkah 2: Pengaturan Database (PostgreSQL)

1. Buka pgAdmin atau alat terminal psql Anda.
2. Buat database baru untuk proyek (misalnya, `assessment_db`).
3. Pastikan Anda mengetahui *username* dan *password* PostgreSQL Anda (misalnya, `postgres` / `password123`).

Langkah 3: Pengaturan Backend & Seeding Akun Admin

```
cd backend
npm install
```

Buat file variabel lingkungan (environment variables) `.env` di dalam folder `/backend`:

```
PORT=8000
DATABASE_URL="postgresql://<DB_USER>:<DB_PASSWORD>@localhost:5432/<DB_NAME>?
schema=public"
JWT_SECRET="rahasia_jwt_anda"
FRONTEND_URL="http://localhost:3000"
```

Hasilkan (*generate*) Prisma Client dan sinkronkan skema ke database PostgreSQL Anda:

```
npx prisma generate
npx prisma db push --accept-data-loss
```

Penting - Pembuatan Akun Admin: Agar Anda dapat login ke dalam sistem untuk pertama kalinya, Anda **wajib** memasukkan data (seeding) ke dalam database yang masih kosong. Jalankan skrip *seed* berikut untuk membuat akun admin secara otomatis:

```
npx ts-node scripts/create_admin.ts
```

Catatan Kredensial Login: Setelah skrip seeding dijalankan, Anda dapat melakukan login ke dalam aplikasi menggunakan data berikut:

- **NIK:** 000000
- **Password:** password123
- **No. Handphone:** 0812345678

Langkah 4: Pengaturan Frontend

```
cd apps/web  
npm install
```

Buat file `.env.local` di dalam `/apps/web` yang menunjuk ke *backend* Anda:

```
NEXT_PUBLIC_API_URL="http://localhost:8000/api"
```

Langkah 5: Pengaturan AI Service

```
cd apps/ai_service
```

Buat Virtual Environment Python:

```
# Pada Windows  
python -m venv .venv  
.\.venv\Scripts\activate  
  
# Pada Mac/Linux  
python3 -m venv .venv  
source .venv/bin/activate
```

Instal dependensi dan siapkan `.env`:

```
pip install -r requirements.txt
```

```
OPENAI_API_KEY="sk-api-key-openai-anda"  
BACKEND_URL="http://localhost:8000/api/ai/analyze-response"  
PORT=8001
```

7. Menjalankan Proyek Secara Lokal

Setelah pengaturan selesai, Anda dapat menjalankan server dengan menggunakan skrip NPM standar atau skrip batch (.bat) yang telah disediakan. Jalankan pada tiga jendela terminal yang terpisah:

Terminal 1: Jalankan Server Backend

Menggunakan skrip .bat:

```
start-backend.bat  
# Untuk menghentikan, Anda dapat menjalankan stop-backend.bat
```

(Atau menggunakan `npm run dev` di dalam folder backend. Berjalan di `http://localhost:8000`)

Terminal 2: Jalankan Server Frontend

Menggunakan skrip .bat:

```
start-frontend.bat  
# Untuk menghentikan, Anda dapat menjalankan stop-frontend.bat
```

(Atau menggunakan `npm run dev` di dalam folder apps/web. Berjalan di `http://localhost:3000`)

Terminal 3: Jalankan AI Service

```
cd apps/ai_service  
# Pastikan virtual environment dalam keadaan aktif  
.\.venv\Scripts activate # Windows  
# source .venv/bin/activate # Mac/Linux  
  
python main.py
```

(Berjalan di `http://localhost:8001`)

Dengan ketiga layanan yang berjalan, sistem akan beroperasi secara penuh secara lokal!